Module 14-1

Introducing the Process of Synthesis

cadence

This module explores the basics of synthesis – how it works, where it fits in your flow, its strengths and weaknesses, and some things to look out for.

Module Objective

In this module, you:

- Briefly examine the synthesis process and its results

Topics

- What is Logic Synthesis?
- How do you use synthesis?
- What does synthesis do?
- What synthesis sometimes can't do well
- Technology-specific issues
- Synthesis in your design flow
- Synthesizable Verilog

Your objective is to be able to briefly describe the synthesis process and its results to team members that are unable to take this course.

To do that you should know at least somewhat about:

- How you use synthesis.
- What the synthesis tool does.
- Synthesis tool strengths and weaknesses.
- Technology-specific issues.
- The Verilog language synthesis "subset".

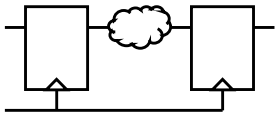
What Is Logic Synthesis?

Logic synthesis is a tool that:

- Infers storage elements (latches, registers).
- Infers combinational logic feeding storage elements.
- Optimizes generated structure to meet cost factors:
  - Area / speed / power / routability

A person new to synthesis might ask:

- How much of the design?
- How accurately?
- How optimally?



cadence

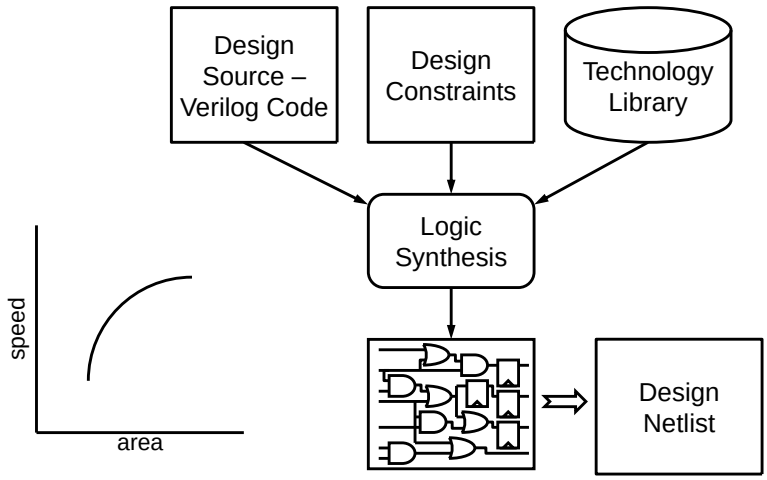
Synthesis automatically transforms your RTL Verilog design into an optimized netlist of predefined cells. Some obvious first questions might be the following:

- What parts of the design can synthesis handle?
- Is the result truly functionally equivalent to the RTL?
- Just how optimal is that result really?

How Do You Use Synthesis?

You provide design source, design constraints, and technology library.

The tool infers logic from the HDL source, maps the inferred logic to technology library macros, and optimizes the circuit to meet constraints.



4

cadence

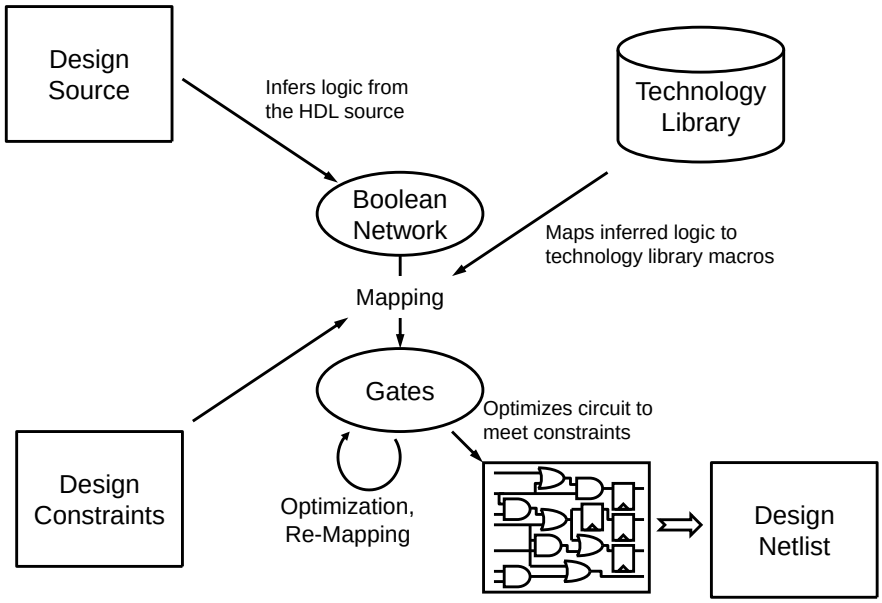
A synthesis tool can read your RTL Verilog code and convert it into a gate-level netlist, using gates and cells from a specified target technology library.

The synthesis tool will try to meet your design constraints for the area and performance, and will produce reports to tell you how successful it was.

Design constraints include the required clock speed, input drive strength and arrival time with respect to the clock, and output load and required arrival time with respect to the clock. Design constraints can also include power consumption and can sometimes include noise immunity, testability, and factors affecting ease of place and route.

The synthesis tool trades circuit speed and circuit area. As a general statement, it can produce a small design or a fast design or a compromise between small and fast. Several factors shift the area/speed curve. Among those factors are the target technology and the circuit architecture.

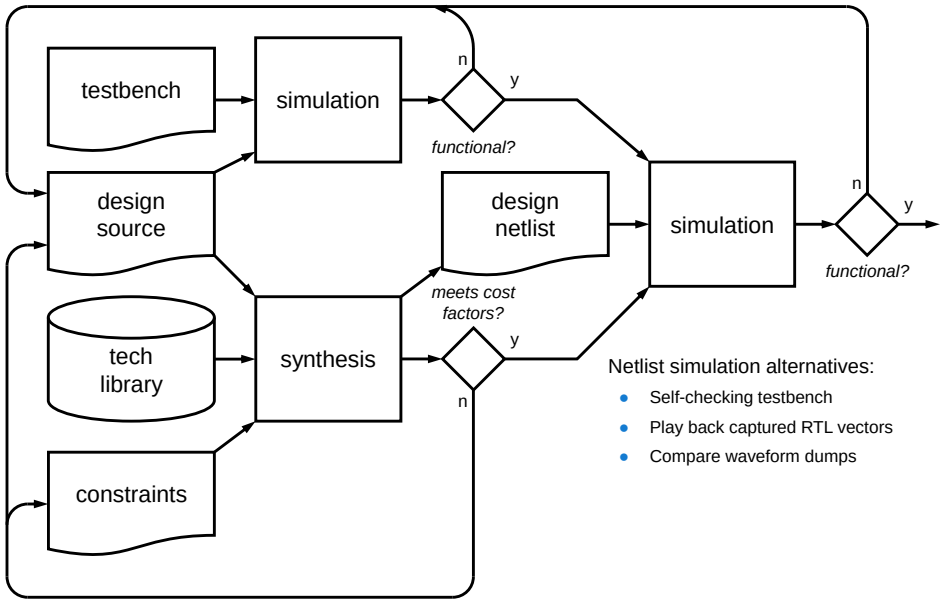
What Does Synthesis Do?



This diagram is of course very simplified. Synthesis technology continues to evolve and this course does not intend to describe its particulars anyway. In a very general sense:

- The tool first transforms HDL input into an internal Boolean network of sequential storage elements and the logic “cone” expressions that feed them.
- The tool then does some logic restructuring, such as to prune logic that does not drive anything and to share common subexpressions. It also at this point makes a guess about operator implementation, basing its choice on expression length. For example, for a add operation it might choose a carry look-ahead adder instead of a ripple-carry adder based on operand width.
- The tool next maps the design to the gates actually available in the target library and performs optimizations based on your constraints and the actually parameters of those gates.

Synthesis in Your Design Flow



This diagram is of course very simplified. Synthesis technology continues to evolve and this course does not intend to describe its particulars anyway. In a very general sense:

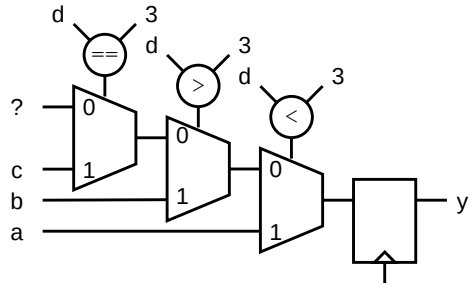
- The tool first transforms HDL input into an internal Boolean network of sequential storage elements and the logic “cone” expressions that feed them.
- The tool then does some logic restructuring, such as to prune logic that does not drive anything and to share common subexpressions. It also at this point makes a guess about operator implementation, basing its choice on expression length. For example, for a add operation it might choose a carry look-ahead adder instead of a ripple-carry adder based on operand width.
- The tool next maps the design to the gates actually available in the target library and performs optimizations based on your constraints and the actually parameters of those gates.

Logic Inference Is Literal

Synthesis tools infer logic from HDL structure and statements.

- They then optimize the logic as needed to meet constraints.
- Here the == operator is redundant but could still appear in the netlist.

```
always @(posedge clk)
  if (d < 3)
    y <= a;
  else if (d > 3)
    y <= b;
  else if (d == 3)
    y <= c;
```



What is the "?" input?

Synthesis tools do not interpret or analyze your code to understand your intention. They initially transform your RTL design to Boolean logic quite literally, and then optimize it. They stop optimization when the design meets your requirements or they run out of time. For any non-trivial design, synthesis produces an optimization that is hopefully “good enough” but not likely to be perfect.

You can help the optimization process by explicitly organizing and coding your design as closely as possible to what you want the optimized result to look like. Write RTL but “think” hardware. A later module discusses such RTL coding best practices for synthesis.

The design as first parsed and elaborated will reflect the RTL code. It will contain a priority structure of multiplexors and operators. If all inputs arrive at the same time, then the optimized design will likely consist of a sum of three products, the priority structure having been optimized.

Coding Style Affects Results

Synthesis tools infer logic from HDL structure and statements.

- Logic synthesis tools do not insert storage elements.
  - Pipeline considerations (number of registers) are totally your responsibility.
- Some logic synthesis tools can move storage elements upstream or downstream in combinational logic to balance delays.
- The quality of results depend mainly on code.

A synthesis tool does not optimize the registers or latches in your design. Be careful how you structure your registered processes so that you only infer the registers you need. Check synthesis reports carefully to make sure.

You also need to be careful how much combinational logic is placed between adjacent register banks. If you place a 32-bit multiplier between adjacent register banks with a 10ns clock period, don't be surprised if the synthesis tool struggles to produce a result.

Generally, you need to be careful how you partition and structure your design into combinational and registered logic.

What Synthesis Sometimes Can’t Do Well

- Clock trees
  - Usually require detailed, accurate net delay information.
- Complex clocking schemes
  - Synthesis tools prefer simple, single clock synchronous designs.
- Memory, IO pads, technology-specific cells
  - You will probably need to instantiate these by hand.
- Specific macro-cells
- Synthesis tools can analyze hundreds of potential implementations in much less time than you need to analyze just one.

Logic synthesis primarily targets for optimization of the logic cones between registers. Some things that require special handling are, for example:

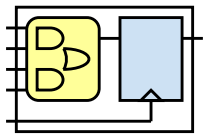
- Clock tree design is a global, chip-wide problem, which particularly in deep submicron designs requires detailed and realistic net delay information. No single clock tree generation method can apply to all designs. You will likely need to design the clock tree later as part of silicon layout.
- Synthesis tools typically have difficulty with complex clocking schemes. Synthesis works most reliably with a single clock per RTL module, and some tools restrict you to at most two clocks or require that the multiple clocks be harmonically related.
- You may have to manually instantiate I/O cells and large regularly structured cells that synthesis tools do not readily select. You may want to utilize the technology vendor’s silicon compilers to generate such large regularly-structured datapath elements and large memory blocks.

For any nontrivial design, synthesis produces an optimization that is hopefully good enough but not likely to be perfect. For a critical path that does not meet timing requirements, it is quite possible that you can manually examine the path and find some way to improve it.

Technology-Specific Issues

- Each technology suggests its own optimal architecture, with architecture-specific “tricks” for good utilization and speed.
- Code becomes technology specific.
- Refer to vendor literature.

ASIC	FPGA
Technologies are similar	Technologies are not similar
Basic units are small and flexible	Basic units are large, not flexible
Optimization algorithms are similar	Optimization algorithms are not similar



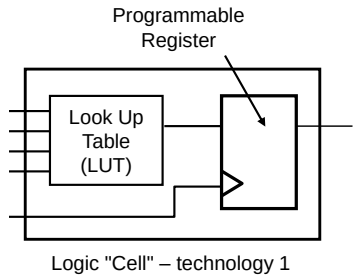
cadence

Each technology suggests its own optimal architecture, with architecture-specific “tricks” for good utilization and speed. To understand these differences, let’s compare synthesis of an ASIC to synthesis of an FPGA:

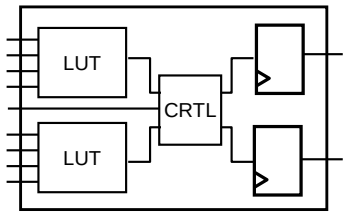
- Synthesis of an ASIC is similar for all technologies – the library cells are relatively small and similar between technologies, routing delays are relatively more predictable, and the tools use basically the same optimization algorithms.
- Synthesis of an FPGA can vary widely between technologies – the FPGA can be EEPROM, SRAM, or anti-fuse, each vendor has different building blocks that tend to be relatively large, and an FPGA architecture is relatively inflexible, leading to relatively widely-varying routing delays. FPGA design employs technology-specific and architecture-specific tricks, implemented as user-defined attributes, comments or direct instantiation, which are difficult for a synthesis tool to implement, thus making your Verilog code technology dependent and reducing its portability.

Programmable Logic Device Specific Issues

- Different architectures for different technologies
- Fixed architecture within a specific technology
- Architecture specific “tricks” for best utilization/speed
- Technology specific/generic code trade-offs
- How your synthesis tool handles your technology



Logic "Cell" – technology 1



Logic "Cell" – technology 2

Many of the comments so far have assumed synthesis to an ASIC. With an FPGA there are some different problems. ASIC synthesis tools all work in the same way, using the same optimization algorithms, and the target technology libraries all have similar cells. With FPGAs there are different technologies, EEPROM, SRAM and anti-fuse, and each vendor has different building blocks, which tend to be much larger than the gate-level primitives used in an ASIC.

For a particular FPGA, the architecture is fixed, limiting the usefulness of static timing analysis since routing delays can vary widely – with an ASIC they are much more predictable. What’s more, to get a good “fit” the FPGA designer traditionally employs architecture specific tricks which are difficult to implement in a synthesis tool.

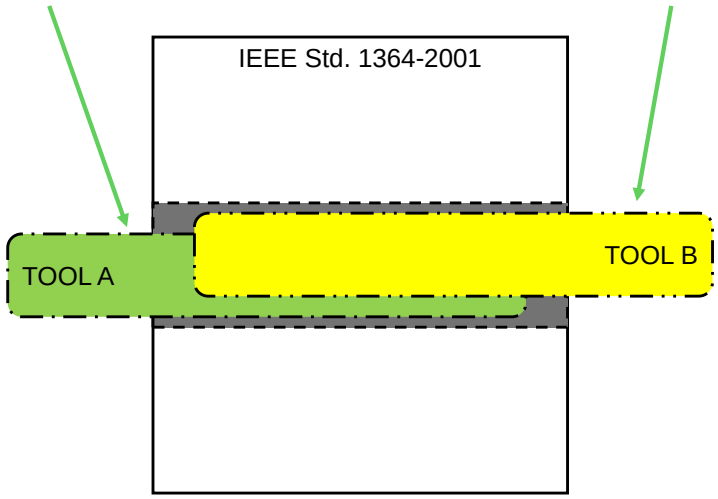
VHDL code may need to make use of technology- and architecture-specific techniques and tricks. These may be expressed with user-defined attributes, comments or direct instantiation of specific cells. Such techniques restrict the technology independence and portability of your code, but allow the most efficient implementation with a specific technology.

A key issue is how well your synthesis tool handles your chosen technology. To achieve a good result, it will need architecture specific algorithms.

Synthesizable Verilog

IEEE Std. 1364.1 defines the synthesizable Verilog constructs.

- Not all synthesis tools yet accept all the standard constructs
- Most synthesis tools still accept also their own legacy constructs



12

cadence

The IEEE Std. 1364.1-2002 for Verilog RTL synthesis describes use of the Verilog "synthesizable subset" in a manner from which logic synthesis tools can infer logic.

- You will rarely but perhaps occasionally use a logic synthesis tool that does not accept some standard synthesizable construct.
- As logic synthesis existed long before the standard, the predominant tools still support constructs that did not become standard.
- This course describes what is standard. If you write your RTL code according to this course, then your code will be portable between the maximum number of synthesis tools.

Module Summary

You should now be able to briefly describe what is synthesis.

This module described:

- What you provide to the synthesis tool:
  - Design source, design constraints, technology library
- What the synthesis tool does:
  - infers logic from the HDL source, maps the inferred logic to technology library macros, and optimizes the circuit to meet constraints
- What the synthesis tool might need help with:
  - Complex clocking schemes, large memories, I/O pads
- What part of Verilog is synthesizable:
  - Defined by IEEE Std. 1364.1

cadence

This module explored the basics of synthesis – how it works, where it fits in your flow, its strengths and weaknesses, and some things to watch for.

Module 14-2

Coding RTL for Synthesis

cadence

This module intends to provide you with the information you need to code your RTL design in compliance with the IEEE Std 1364.1-2002 for Verilog RTL synthesis.

Module Objective

In this module, you:

- Code design behavior for logic synthesis

Topics

- Modeling combinational logic
- Modeling sequential logic
- Modeling latch logic
- Modeling three-state logic
- Using synthesis attributes

Your objective is to code design behavior for logic synthesis.

To do that, you need to know about:

- Modeling combinational logic;
- Modeling sequential logic;
- Modeling latch logic;
- Modeling three-state logic; and
- Using synthesis attributes.

Modeling Combinational Logic

Combinational Logic: Output is at all times a combinational function solely of the inputs.

- As a **net declaration** assignment
- As a **continuous** assignment
- As an **always** statement

```
wire w = expression;
```

```
wire w; assign w = expression;
```

```
reg r; always @* r = expression;
```

  - The event list must not contain a posedge or negedge event.
    - Include all procedure inputs to avoid mismatch between pre-synthesis and post-synthesis designs.
  - Assignments must be blocking, they are sufficient and simulate more efficiently.

```
always @( all block inputs )
  begin
    blocking assignments
  end
```

16

cadence

You can model combinational logic with a net declaration assignment or a continuous assignment or an always statement.

When using an always statement, the single event list must not contain a posedge or negedge event. The event list does not otherwise affect the synthesis result, but to avoid incorrect RTL simulation results, you should include in the event list all inputs to the procedure. The easiest way to ensure this is to use the Verilog 2001 wildcard event control (@*).

You must not in an always statement assign a variable using both a blocking assignment (=) and a nonblocking assignment (<=). The blocking assignment is sufficient for a description of combinational logic and simulates more efficiently than the nonblocking assignment.

“Combinational logic shall be modeled using a continuous assignment or a net declaration assignment or an always statement. ... A variable assigned in an always statement shall not be assigned using both a blocking assignment (=) and a nonblocking assignment (<=) in the same always statement. ... The event list for a combinational logic model shall not contain the reserved words posedge or negedge.” – IEEE Std 1364.-2002 Section 5.1 Modeling combinational logic

Incomplete Event List

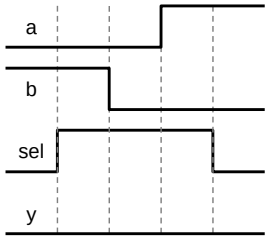
- For simulation, include all in the event list all signals that are input to the logic.
- The `@*` wildcard sensitivity list automatically includes all input signals to the logic.

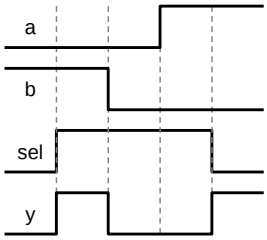
```
// Incomplete list

always @(a or b)
  begin
    y = a;
    if (sel)
      y = b;
  end
```

```
// Complete list

always @*
  begin
    y = a;
    if (sel)
      y = b;
  end
```





The event list does not affect the synthesis result, but to avoid incorrect RTL simulation results, you should include in the event list all inputs to the procedure. The easiest way to ensure this is to use the Verilog 2001 wildcard event control (@*).

This example illustrates the affect of an incomplete sensitivity list.

- If the event list omits the sel signal, the procedure executes upon transitions of only the a and b inputs – transitions of the sel signal have no effect.
- Debugging problems caused by an incomplete sensitivity list is difficult, so you might want to develop the habit of simply always using the wildcard event control for all combinational procedures.

“The event list does not affect the synthesized netlist.” – IEEE Std. 1364.1-2002 5.1 Modeling combinational logic

Complete Event List

Do not include temporary variables – those written and then read in the same procedure and nowhere else.

```
// Combinational logic

always @(a or b or c)
  q = a + b + c;
```

The event list for a combinational procedure should contain all inputs to the logic.

```
// Combinational logic

always @(a or b or c)
  begin : comb_blk
    reg temp;
    temp = a + b;
    q = temp + c;
  end
```

Do not place temporary variables in the sensitivity list.

The Verilog language lets you place any signals in the sensitivity list and to freely mix blocking and nonblocking assignments anywhere in the procedure. A simulation tool simply executes the procedure as you wrote it, using simulation semantics. However, to generate RTL simulation results consistent with those of the postsynthesis netlist, some guidelines exist:

- All edges of all signals input to combinational logic must be present in the sensitivity list. This is due to the rule that any changes on any inputs to the logic must have the opportunity to immediately affect the output. Do not place temporary variables in the sensitivity list. A temporary variable is one that the procedure writes only before it reads and that no other procedure uses. It is not an input to the logic.
- Do not mix blocking and nonblocking assignments to the same variable. Even better, you should use only blocking assignments within procedures that represent purely combinational logic. This is a recommendation to obtain higher simulation performance, as nonblocking assignments are not necessary and simulate more slowly.

The synthesis tool requires code that unambiguously states your design intentions. Code meant for the synthesis tool may use only a subset of the Verilog constructs and coding styles.

- For synthesis, it is an absolute requirement that you do not mix blocking and nonblocking assignments to the same variable!
- The synthesis standard states that the sensitivity list shall not affect the generation of combinational logic. That means that if the synthesis tool recognizes the block as combinational logic, it will proceed as if you had included all inputs to the logic in the sensitivity list. The synthesis tool may or may not warn you about missing inputs. The generated gates will simulate correctly, but very likely differently than the incorrect RTL simulation.

Incomplete Assignments

Combinational Logic

- Output is at all times a combinational function solely of the inputs.

```
// Incomplete assignment

always @(a or b)
  if (b)
    y = a;
```

```
// Incomplete assignment

always @(a or b)
  case (b)
    1: y = a;
  endcase
```

What is the value of `y` when `b` is 0?
How does logic synthesis implement this behavior?

If an execution path through a combinational procedure exists that does not update the value of some output, then that output variable must retain its previous value. The synthesis tool infers a latch to implement this behavior. Latch inference is almost always not intended, and you can easily avoid it.

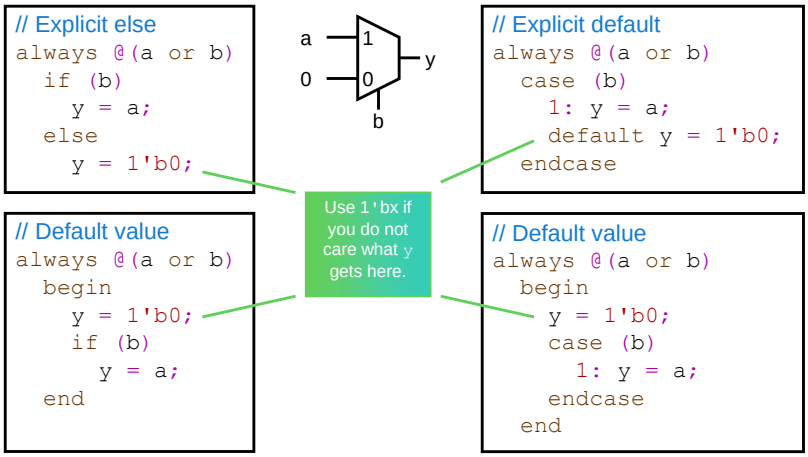
This example fails to update the y output variable when the b input is not 1.

How would you modify this code to ensure inference of purely combinational logic?

Complete Assignments

Avoiding latch inference is easy!

- Provide outputs with a value for every combination of inputs.
- This is most easily done with default assignments.



Here are two methods you can use to prevent latch inference:

- You can for an if statement use an explicit else clause and for a case statement an explicit default match item; or
- You can provide default values for all procedure outputs at the start of the procedure.

Which is the best technique in a real design?

If you have a procedure with a complex set of conditional assignments, you can easily miss an assignment for one or more of these branches. Making default assignments at the start of the procedure ensures that all procedure outputs have an assignment.

“The value x may be used as a primary on the RHS of an assignment to indicate a don’t care value for synthesis.” – IEEE Std. 1364.1-2002 Section 5.5 Support for values x and z

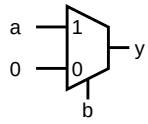
Continuous Assignments

Continuous assignments drive values onto nets.

Continuous assignments always represent combinational logic.

- Impossible to not assign a value for some input combination.
- Output is always a combinational function of current inputs.

```
assign y = b ? a : 1'b0;
```



Continuous assignments and net declaration assignments drive values onto nets. These assignments always represent combinational logic, as it is syntactically impossible to not assign a value for some input combination, thus the output is always a combinational function of the current inputs.

Modeling Combinational Logic Summary

Summary of the steps to procedurally describe purely combinational logic:

- Start the procedure with the **always** construct.
- Immediately follow the always construct with an event control.
  - Place all possible input events in the event expression.
- Group multiple following statements within a **begin...end** block.
- Use blocking assignments.
- Provide default assignments to prevent latch inference.
- Avoid combinational feedback loops.
- Assign a variable in only one procedure.

- Continuous assignments always synthesize to combinational logic.
- Subroutines almost always synthesize to combinational logic.

Here is a summary of the steps to procedurally describe purely combinational logic:

- Second level
- Start the procedure with the always construct and immediately follow the always construct with an event control, placing all possible input events in the event expression.
- Group multiple following statements within a begin...end block. You can omit the begin and end keywords if you have only a single statement.
- Make blocking assignments. You can equally validly make nonblocking assignments, but they simulate less efficiently and provide no additional benefit. In any case, do not mix blocking and nonblocking assignments to the same variable and do not make assignments to a variable from multiple procedures.
- Provide default assignments at the start of the procedure to prevent latch inference.
- Avoid combinational feedback loops.

Remember that:

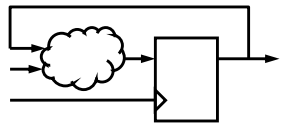
- Continuous assignments always synthesize to combinational logic.
- Functions almost always synthesize to combinational logic.

Modeling Sequential Logic

Sequential Logic

- Outputs are sampled in registers on a clock edge, thus storage is required.
- As an always statement:
  - The event list must contain only posedge and negedge events.
    - One represents the active clock edge and others represent asynchronous set and reset.
  - Model set/reset behaviors in an if statement early branches and normal behavior in the later branches.
  - Make nonblocking assignments to storage variables.

```
always @( clock edges )
  begin
    nonblocking assignments
  end
```



You model sequential logic using an always statement that has one or more posedge or negedge events in exactly one event list. Exactly one of those events represents the active clock edge that stores the value. Any additional posedge or negedge events represent asynchronous set and reset behaviors. The event list must not contain level-sensitive events.

You model the asynchronous set and reset behaviors by using an if statement. In the if statement, you model the asynchronous behaviors in one or more conditional branches. The unconditional last else branch models the normal sequential behavior.

To avoid simulation clock/data races, you should make only nonblocking assignments to variables that represent storage. You make blocking assignments to temporary variables. Temporary variables are those written and then read in the same procedure and nowhere else.

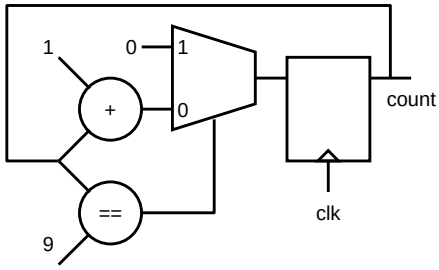
Normal Behavior

The event list for a sequential procedure must contain only single edges of clock signals.

- All events must be posedge or negedge qualified.
- The synthesis tool infers registers.
- For non-temporary variables assigned in sequential procedures:
  - Assigned with non-blocking statements.

```
// Sequential logic

always @(posedge clk)
  if (count == 9)
    count <= 4'd0;
  else
    count <= count + 4'd1;
```



Something seems not quite right here...

Synthesis tools recognize sequential procedures by looking for a particular code template – in this case, a procedure with an event list containing only edge-qualified signals. Although some synthesis tools may support other coding styles for sequential procedures, your adherence to this standard produces code that you can port between all synthesis tools compliant with the standard.

This example has only the positive edge of the clock in its event list. All storage inferred for non-temporary variables that this procedure writes will have a rising active clock edge. The if statement describes the combinational logic calculating the new “count” value that is stored on the next rising clock edge.

Note that this example lacks a reset to initialize the count value!

Reset Behavior

Use an **if...else** statement to add set / reset to a procedure.

- Put the set / reset behavior in the first branch.
- Put the normal sequential behavior in the last branch.
- For asynchronous resets, add active set / reset edges to event list.

```
// Asynchronous reset

always @(posedge clk
      or posedge rst)
  if (rst)
    count <= 4'd0;
  else
    if (count == 9)
      count <= 4'd0;
    else
      count <= count + 4'd1;
```

```
// Synchronous reset

always @(posedge clk)
    // all else is same
  if (rst)
    count <= 4'd0;
  else
    if (count == 9)
      count <= 4'd0;
    else
      count <= count + 4'd1;
```


cadence

You can most easily provide set and reset behaviors by using an if statement. This is a requirement for asynchronous set and reset behaviors and strongly recommended for synchronous set and reset behaviors. The synthesis tool will treat synchronous set and reset behaviors you model outside an if statement as just that much more combinational logic.

In the if statement, place the set and reset behaviors in their order of priority in the first conditional branches, and place the normal synchronous behavior in the unconditional last else branch. Do not make assignments to the storage variable outside of the if statement.

For synchronous set and reset, trigger the procedure on only the clock edge.

For asynchronous set and reset, trigger the procedure also on the active set and reset edge(s).

Code synchronously reset, asynchronously reset and not reset registers in separate procedures.

Sequential Procedure Templates

Code must follow these templates:

- Procedure starts with an always construct.
- Followed by one event control containing only edge-qualified signals.
  - Clock
  - Asynchronous set or reset

```
always @(posedge clk)
  begin
    // normal
    // sequential
    // behavior
  end
```

```
always @(posedge clk)
  if (!rst)
    begin
      // synchronous
      // reset
      // behavior
    end
  else
    begin
      // normal
      // sequential
      // behavior
    end
```

```
always @(posedge clk
      or negedge rst)
  if (!rst)
    begin
      // asynchronous
      // reset
      // behavior
    end
  else
    begin
      // normal
      // sequential
      // behavior
    end
```


Sequential procedures have the clock edge in the event list, and also the set and reset edge(s) if there is asynchronous set or reset.

Separately code your not reset, synchronously reset and asynchronously reset procedures.

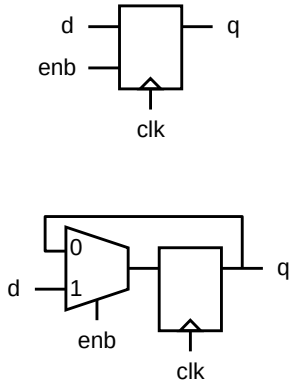
Incomplete Assignments

Sequential Logic

- Output is not at all times a combinational function solely of the inputs.
  - Implies some sort of storage.
- Incomplete assignment in a sequential procedure does not infer a latch.
  - Storage is already there!

```
always @(posedge clk)
  if (enb)
    q <= d;
```

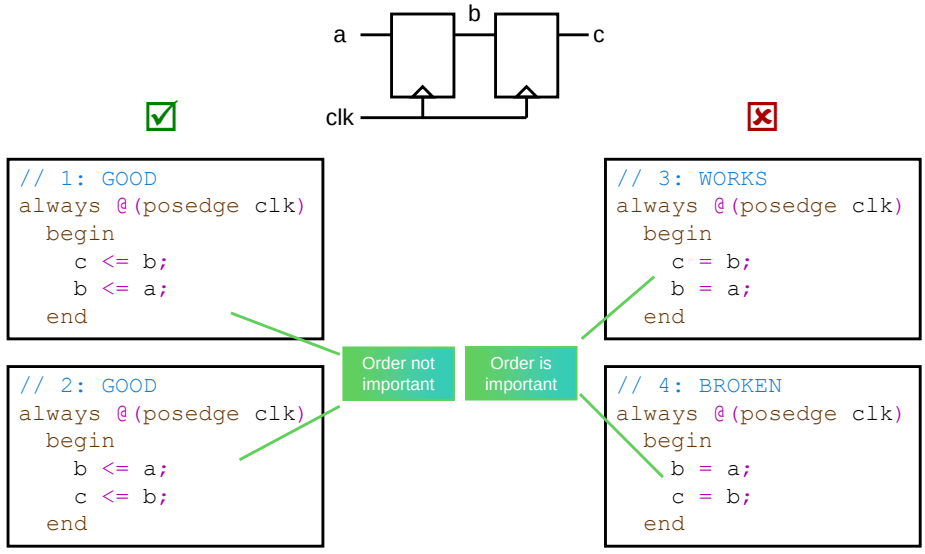
Potential Implementations



If procedure execution does not update a variable, then the variable value is not changed. In procedures representing combinational logic, this infers a latch to store the previous variable value. For procedures representing synchronous logic, the variable value is stored in the inferred register. For these procedures you do not use default assignment or else clauses to prevent latch inference.

Blocking vs. Nonblocking Assignment

Write a Verilog sequential procedure to codify this behavior.



Good coding practice demands that you make only nonblocking assignments to variables that represent storage elements. This is to avoid clock/data races in simulation and to avoid unintended logic inference in synthesis. The synthesis standard requires only that you not mix the two types of assignment to the same variable.

For examples 1 and 2, you can see that with nonblocking assignments the order of assignments is not important, as simulation schedules the actual assignments for the NBA region of the stratified event queue.

For examples 3 and 4, you can see that with blocking assignments the order of assignments is important, as simulation completes the assignments as it executes the statements. Different order generates different simulation results and different synthesis results.

- Example 3 reads variable b before writing it, so synthesis infers two registers;
- Example 4 reads variable b only after writing it. Variable b is a temporary variable. Synthesis infers one register.

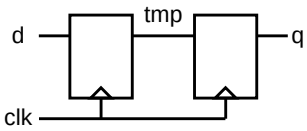
“Nonblocking procedural assignments should be used for variables that model edge-sensitive storage devices.” – IEEE Std. 1364.-2002 Section 5.2.2 Modeling edge-sensitive storage devices

Temporary Variables in Sequential Procedures

Temporary Variable

- Written first and then read (in same procedure).
- Variable is alias for expression so no register is inferred.

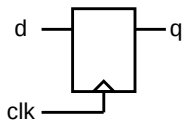
```
// 2 flop
always @(posedge clk)
  begin: dff2
    reg tmp;
    q <= tmp;
    tmp = d;
  end
```



Persistent Variable

- Read first and then written (in same procedure).
- Synthesis must infer a register to hold the value to the next read.

```
// 1 flop
always @(posedge clk)
  begin: dff1
    reg tmp;
    tmp = d;
    q <= tmp;
  end
```



cadence

You can use a temporary variable to hold the intermediate result of a calculation before you use the result later in the procedure. You use temporary variables to break up complex expressions and combinational logic into a series of smaller steps – making complex logic easier to describe, understand and maintain.

A temporary variable is one that a procedure writes with a blocking assignment only before the procedure reads it, and no other procedure uses it. You can declare temporary variables locally to ensure that no other procedure uses it.

A typical use model would be to make blocking assignments to temporary variables and then make nonblocking assignment of the temporary variable values to other variables.

“Blocking procedural assignments may be used for variables that are temporarily assigned and used within an always statement.” – IEEE Std. 1364.-2002 Section 5.2.2 Modeling edge-sensitive storage devices

Modeling Sequential Logic Summary

Summary of the steps to procedurally describe sequential logic:

- Start the procedure with the *always* construct.
- Immediately follow the always construct with an event control.
  - Use only posedge or negedge events in the event expression.
  - Include only clock and asynchronous set / reset events.
- Group multiple following statements within a *begin...end* block.
- Place the asynchronous set / reset behavior first (in their order of priority) in the *if* statement.
- Place the normal sequential behavior in the last else branch.
  - Make blocking assignments to only the temporary variables.
    - Write temporary variables before and not after they are read.
  - Make nonblocking assignments to the register variable.
- Do not make assignments to a variable from multiple procedures.

Here is a summary of the steps to procedurally describe sequential logic:

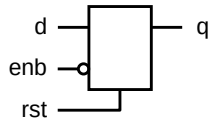
- Start the procedure with the always construct and immediately follow the always construct with an event control, placing only edge-qualified events in the event expression.
- Group multiple following statements within a begin...end block. You can omit the begin and end keywords if you have only a single statement.
- Make only blocking assignments to temporary variables and make only nonblocking assignments to variables representing storage. Write the temporary variables only before you read them in the same procedure. Do not make assignments to a variable from multiple procedures.

Modeling Latch Logic

But what if you want to infer a latch?

- A combinational block that for some combination of inputs does not provide an output value infers storage, i.e., latch.
- Make assignments for latch logic as you do for sequential logic.
  - Make blocking assignments to only the temporary variables.
    - Write temporary variables only before they are read.
  - Make nonblocking assignments to the latch variable.

```
always @(enb or rst or d)
  begin: latch
    reg tmp;
    tmp = d;
    if (rst)
      q <= 0;
    else
      if (enb)
        q <= tmp;
  end
```



Latch inference is almost always a mistake – be sure to document where and why you did it!

cadence

You deliberately infer a latch, if you truly want to, the same way you inadvertently infer a latch, with the exception that you remember to make nonblocking assignments to the variable that represents the storage.

A latch-based design can have higher performance than a register-based design, but can also be more difficult to correctly design and debug. Use of latches is typically infrequent and reserved for extenuating circumstances.

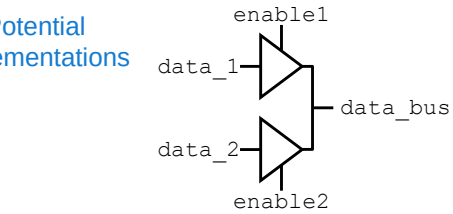
Modeling Three-State Logic

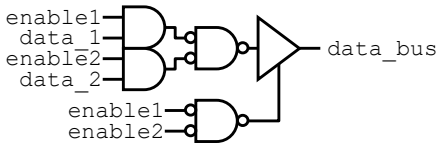
Synthesis infers three-state logic when you assign a net or variable the high-impedance value.

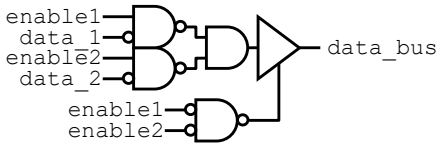
```
wire data_bus;
assign data_bus = enable1 ? data_1 : 8'bz;
assign data_bus = enable2 ? data_2 : 8'bz;
```

```
reg data_bus;
always @*
  begin
    if (enable1)
      data_bus = data_1;
    else
      data_bus = 8'bz;
    if (enable2)
      data_bus = data_2;
    else
      data_bus = 8'bz;
  end
```

Potential Implementations







The standard does not specify an implementation.

Ensure the enable signals are mutually exclusive!

cadence

You model three-state logic by assigning the high-impedance (z) value to a net or variable. Any other assignments to that net or variable must also assign the high-impedance value. Further propagating the high-impedance value by use of net or variable assignments does not also make that downstream logic into three-state logic.

This example codes behavior representing three-state drivers in a form that synthesis can recognize. For this example:

- The “enable1” signal when high drives “data_1” onto the data bus; and
- The “enable2” signal when high drives “data_2” onto the data bus.

The synthesis standard does not address what the result should be if multiple enables are simultaneously true.

“Three-state logic shall be modeled when a variable is assigned the value z.” – IEEE Std 1364-2002 Section 5.4 Modeling three-state drivers.

Reference: Unsupported Verilog Constructs

Synthesis tools compliant with IEEE Std. 1364.1 do not support:	
assign/deassign (as a procedural statement)	repeat
defparam	time
disable	tri0/tri1/trireg
event	cmos/nmos/pmos/rcmos/rnmos/rpmos
force/release	tran/tranif0/tranif1/rtran/rtranif0/rtranif1
forever	wait
fork/join	while
macromodule	-> (event emission)
primitive	===/!=== (case identity)
pulldown/pullup	expression in event list
real	hierarchical identifiers
realtime	system functions (except $signed/$unsigned)
	** (power operator, except as power of 2)

Synthesis tools compliant with IEEE Std. 1364.1 ignore:	
#delay (if occurs after event control)	system tasks
initial (except supported for ROM modeling)	variable declaration assignment
specify (entire block)	`celldefine, `endcelldefine, `line, `timescale, `unconnected_drive,
strength specifications	`nounconnected_drive

Here for your reference, are constructs that the synthesis tool does not support, and constructs that the synthesis tool ignores.

Synthesis supports the always keyword only with an immediately following event control (@).

Module Summary

You should now be able to properly code hardware for logic synthesis.

This module discussed the following:

- Modeling combinational logic
  - In an **always** block with complete sensitivity list, blocking assignments, and all block outputs assigned each time the block is executed.
- Modeling sequential logic
  - In an **always** block sensitive to edge-qualified events, nonblocking assignments, and set/reset behavior in early conditional statement branches.
- Modeling latch logic
  - By deliberately omitting an output value for some combination of input values.
- Modeling three-state logic
  - By assigning the high-impedance (Z) state when disabled.

This module explained how to describe hardware for logic synthesis. It addressed combinational, sequential, latch, and three-state logic. It introduced synthesis attributes that you embed in the source code to influence the synthesis process.

Module Exercise

Fix these procedures to make them legal for synthesis:

```
// 1
always @(posedge clk)
  if (clk)
    q <= d;
  else
    if (rst)
      q <= 0;
```

```
// 2
always @(posedge clk
      or rst)
  if (rst)
    q <= 0;
  else
    q <= d;
```

```
// 3
always @(posedge clk
      or negedge rst)
  if (rst)
    q <= 0;
  else
    q <= d;
```

```
// 4
always @(posedge clk
      || posedge rst)
  if (rst)
    q <= 0;
  else
    q <= d;
```

```
// 5
always @(posedge clk)
  if (!rst)
    q <= d;

always @(posedge rst)
  q <= 0;
```

```
// 6
always @(posedge clk
      or posedge rst)
  begin
    if (clk)
      q <= d;
    else
      q <= 0;
    if (rst)
      q <= 0;
    else
      q <= d;
  end
```

Refer to the synthesis templates

cadence

This page does not contain notes.

Module Exercise Solution

Fix these procedures to make them legal for synthesis:

```
// 1
always @(posedge clk)
  if (/*clk*/rst)
    q <= /*d*/0;
  else
 // if (rst)
      q <= /*0*/d;
```

```
// 2
always @(posedge clk
      or posedge rst)
  if (rst)
    q <= 0;
  else
    q <= d;
```

```
// 3
always @(posedge clk
      or negedge rst)
  if (~rst)
    q <= 0;
  else
    q <= d;
```

```
// 4
always @(posedge clk
/*||*/or posedge rst)
  if (rst)
    q <= 0;
  else
    q <= d;
```

```
// 5
always @(posedge clk)
  if (/*!*/rst)
    q <= /*d*/0;
  else q <= d;
//always @(posedge rst)
// q <= 0;
```

```
// 6
always @(posedge clk
      or posedge rst)
  begin
 // if (clk)
 //   q <= d;
 // else
 //   q <= 0;
    if (rst)
      q <= 0;
    else
      q <= d;
  end
```


cadence

This page does not contain notes.